

Business Entity Resolution

ML Challenge 2026 — Team Handover & Progress Report

Status: full-scale train+test run in progress (launched today, local machine).

This document captures everything built and learned so far, in enough detail to hand off and extend.

Prepared for the team, from the working session building and running the pipeline.

1. Executive Summary

We built a complete, rules-plus-classical-ML entity resolution pipeline (no LLMs, per the challenge's fair-play rules) that matches noisy S2/S3 business records to clean S1 reference entities across US, India, and (test-only) France. The pipeline is a Python package (**business_entity_resolution/**) with independently cached, resumable stages: **normalize** → **block** → **featurize** → **train** → **tune** → **predict** → **validate**. It runs unmodified locally, on Kaggle, or on Colab.

On a 5% held-out dry-run subsample, the pipeline scores **F0.5 = 0.9390** (macro-averaged, matching the challenge's exact scoring formula), verified with a 2-fold cross-fit estimate (0.9391 — i.e. no overfitting to the tuning process) and beating a simple interpretable weighted-sum baseline by +0.0126. We diagnosed a real, quantified performance gap between India and US (India's blocking recall ceiling and final F0.5 both trail US's meaningfully), attempted two targeted fixes, found both made things *worse* on rigorous ablation testing, and reverted both — full detail and reasoning in Section 8, since this is exactly where the next improvement should start.

As of writing, a **full-scale run (all ~2.2M train + ~1.7M test S1 entities) is running locally**, using the confirmed-good reverted configuration. Section 9 has the live status, exact commands, and how to check on it.

2. The Problem (as given)

Given business records from 3 sources — S1 (clean, deduplicated reference), S2 and S3 (noisy, independent) — find every S2/S3 record that refers to the same real-world business as each S1 entity. A S1 entity may match zero, one, or many S2/S3 records.

Columns: entity_id (prefixed S1-/S2-/S3-), business_name, business_address, country. Country is US/India in training; test adds France, which the model has never seen labels for.

Scoring: $F_{0.5} = 1.25 \cdot P \cdot R / (0.25 \cdot P + R)$, computed **per S1 entity** then macro-averaged over all entities. A singleton (no true matches) scores 1.0 for an empty prediction and 0.0 for any non-empty one. The 0.5 exponent means **precision is weighted 2× over recall** — a false merge is punished harder than a missed match.

Constraints: no LLMs; only classical ML and embeddings; final model must be MIT/Apache-licensed and ≤8B parameters; no external data lookups (geocoders, registries, APIs) — every normalization rule had to come from patterns in the provided data itself.

3. Dataset Facts (from EDA)

3.1 Scale

Split	S1	S2	S3	Total records
Train	2,206,821	5,034,616	5,285,603	~12.5M
Test	1,732,544	4,887,274	5,082,317	~11.7M

3.2 Ground truth statistics (train)

- Mean 3.46 matches per S1 entity, max 11. Only **5.6% of S1 entities are singletons** (no true match).
- Links split roughly 48% S2 / 52% S3.

- **No S2/S3 record is ever claimed by more than one S1** (confirmed: 0 out of 7.64M links). This is a load-bearing fact for the whole design — it means a S2/S3 record has one true owner at most, so a greedy "give the candidate to whichever S1 scored it highest" assignment is provably at least as good as thresholding first and resolving collisions after.
- About 26% of S2/S3 records are unlinked distractors (they don't match any S1 at all).

3.3 Noise patterns observed directly in the data

- **Native script:** 11-15% of S2/S3 names are in Devanagari/Bengali/Odia script, often transliterated rather than translated (e.g. "Seven Logistics Limited" appears as "■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■").
- State names appear as native script, as codes (RJ/KA/MH), or as English names (North Carolina vs NC)
- City aliases: Bangalore/Bengaluru, Calcutta/Kolkata, Poona/Pune, etc.
- Accent injection ("Léarning", "Prívate"), junk wrapper characters (--, <<, ##, brackets), trailing "| www.x.com", and domain names used as the business name itself (e.g. "surgicalcareassociates.com").
- dba/trading-as clauses, repeated tokens ("LLC LLC"), legal suffixes moved to the front ("LLC Moncada Learning Center"), honorifics ("The", "Shri", "Dr").
- Address noise: perturbed house numbers ("005101", "408-410", "E-11/8" vs "E-1/8"), ordinal variants ("10TH" vs "Tenth"), concatenated words ("WORTHINGTONCITY"), literal "null" tokens, ~3.3% of S2/S3 records have a fully empty address.
- Landmark phrases in ~5% of addresses ("Near Fortis Hospital"), including Hindi postpositions ("... Ke Pas").
- Pincodes are rare (~1% have a clean 6-digit Indian PIN); most 5-digit numbers are house numbers, not ZIP codes.

4. Pipeline Architecture

Everything lives under **business_entity_resolution/**, structured to run as a script locally or be pasted cell-by-cell into a Kaggle/Colab notebook (via `notebook_cells.py` / `make_notebook.py`).

```
business_entity_resolution/
run_pipeline.py          CLI: --split train|test --stage <name> [--force] [--allow-long]
validate_predictions.py  standalone F0.5 scorer against a held-out ground truth
make_dryrun_subset.py   builds a 5% stratified subsample for fast iteration
notebook_cells.py       the pipeline as "# %% [n] SECTION" notebook cells
make_notebook.py        notebook_cells.py -> pipeline.ipynb (stdlib json only)
run_full.sh             the exact script currently running the full train+test job
requirements.txt / requirements-colab.txt
README.md
tests/test_smoke.py
src/ber/
  config.py             paths, GPU/CPU auto-detect, every tunable knob (env-var overridable)
  io.py                 UTF-8 + tab-separated + "\n"-only-line-ending TSV read/write
  lexicons.py           every hand-authored normalization dictionary
  normalize.py          Section 5 below
  embed.py              sentence-transformer embeddings, chunk-resumable
  ann.py                FAISS index build/search (GPU when available)
  tfidf.py              char n-gram TF-IDF for blocking re-rank + a similarity feature
  blocking.py           Section 6 below
  features.py           Section 7 below
  split.py              deterministic hash-based train/val/calib split by S1 entity
  model.py              labeling, hard-negative sampling, XGBoost, isotonic calibration
  tune.py               Section 8's assignment + threshold logic
```

metrics.py	the exact F0.5 formula, shared everywhere it's used
baseline.py	the simple weighted-sum comparison baseline
output.py	TSV writers + official validator subprocess wrapper
bench.py	the --stage benchmark projections
pipeline.py	stage orchestration + on-disk resume caching

Stage caching: every stage checks whether its cached artifact (a parquet file, the model files, etc.) already exists and skips recomputation unless --force is passed. The embedding stage is additionally checkpointed *within itself*, chunk by chunk, so a killed run resumes instead of restarting from zero. This mattered in practice — see Section 9.

5. Normalization Module (src/ber/normalize.py, lexicons.py)

Turns raw (business_name, business_address, country) into normalized strings plus extracted structured fields. Every dictionary was hand-authored from patterns in the provided training data — no external lookups, per the fair-play rules.

5.1 Name normalization

- Unicode NFKC normalize → map known native-script legal/state words to English (■■■■■■■■→limited, ■■■■■■■■→private, ■■■■■■■■→RJ, etc.) → transliterate anything still non-ASCII via **anyascii** (ISC-licensed, not GPL unicode) → strip residual combining marks → casefold.
- Strip junk wrapper characters, split off domain suffixes (setting an is_domain flag), split off dba/trading-as clauses into a separate name_alt field, drop honorifics, collapse immediate repeated tokens.
- Canonicalize legal suffixes to one short token each (Private/Pvt→pvt, Limited/Ltd→ltd, Corporation/Corp→corp, plus llc/llp/co/pc and French forms sarl/sas/sasu/eurl/sa/snc) → produces name_norm (full), name_core (legal tokens stripped), and a legal_forms set.

5.2 Address normalization

- Country-specific abbreviation dictionaries (US: rd/st/ave/blvd/apt/unit/direction words; India: nagar/marg/bypass/opp/nr/sector/phase/colony; France: av/bd/chemin/impasse/route), each variant mapping to one canonical short token so both sides of a match collapse identically regardless of which variant the source used.
- Ordinal-word normalization ("Tenth"→"10th"), null-token stripping ("null"/"none"/"nan"), leading-zero stripping on numeric tokens.
- Extraction into separate fields: pincode/ZIP (country-specific regex), house number (first numeric token), a set of all numeric tokens, state code (from name, code, or native script), city (via an alias dictionary), and landmark phrases ("Near X", "Opp X", Hindi "X Ke Pas") — removed from the main address string into their own field.

5.3 The two forms of each text (important for Section 8)

embed_text = name_core + addr_core (the transliterated/canonicalized ASCII form — used for phonetic keys, RapidFuzz features, TF-IDF, and, currently, embeddings). embed_text_native = a lighter-touch, non-transliterated version that preserves the original script (only built for records with actual non-ASCII content) — currently computed but **not used**; see Section 8.1 for why and how to re-enable it.

6. Blocking / Candidate Generation (src/ber/blocking.py, embed.py, ann.py, tfidf.py)

This stage determines the recall ceiling of the whole pipeline — nothing the classifier does later can recover a true match that never became a candidate. All four blockers, and the re-rank, are scoped **within country** (verified: true matches never cross countries in the ground truth).

6.1 The four blockers, unioned

- **Embedding ANN** (embed.py, ann.py): paraphrase-multilingual-MiniLM-L12-v2 (Apache-2.0) embeddings, L2-normalized, FAISS IVF-SQ8 index (GPU when available via faiss-gpu-cu12, CPU fallback via faiss-cpu — no official Windows faiss-gpu wheel exists, so this runs CPU-only locally). **Forward search:** each S1 queries the S2 index and the S3 index separately, top k=20 each. **Reverse search:** each S2/S3 record queries the S1 index, top k=3 — this recovers S1 entities with many matches (up to 11) that a per-S1 top-k forward search alone could truncate away.
- **Phonetic:** bucket by (state, metaphone-of-first-name-token), then keep the 10 best RapidFuzz token_set_ratio matches within each S1's bucket.
- **House-number:** bucket by (state, house-number), then keep the 10 best RapidFuzz address matches within the bucket. State is folded into the key for the same reason as phonetic — see the bug note in Section 8.3.
- **Sorted-neighborhood:** two passes (raw name_core order, and alphabetically-sorted-token order), sliding window of 5 on each side.

6.2 Bucket-size safety caps

Both the phonetic and house-number blockers cap any bucket that grows past a configurable size (500 by default) by scoring against a capped, deterministic random sample instead of the whole bucket — otherwise a common metaphone code or a common house number like "1" would make a bucket blow up to thousands of candidates, and cost would scale with corpus size *twice over* (more S1 rows to score, and a bigger bucket to score each one against). ANN and sorted-neighborhood cover the resulting recall gap on pathologically generic buckets.

6.3 Union, re-rank, truncation

- The four blockers' outputs are unioned per (S1, candidate) pair with provenance flags (from_ann/from_phonetic/from_sn/from_housenum) and the max embedding similarity kept across any duplicate hits.
- The union is re-ranked by **char 3-4-gram TF-IDF cosine** (a HashingVectorizer, 2^{20} features, IDF fit once on train and reused as-is for test so test document frequencies never leak into weighting).
- Truncated to the top k_final candidates per S1 (chosen automatically from a recall-ceiling sweep at $k \in \{5, 10, \dots, 50, \text{full_union}\}$ — the smallest k within 0.002 of the full union's ceiling F0.5; landed on **k_final=40** on the dry run).
- Per-candidate and per-S1 **competition features** are computed on this final truncated set: how many S1s want this same S2/S3 record, this pair's rank among that record's suitors, and this pair's rank among the S1's own candidates.

6.4 Recall sweep measured on the dry run

k	recall	ceiling F0.5
5	0.889	0.970
10	0.949	0.982
20	0.960	0.985
40 (chosen)	0.965	0.988
full_union	0.969	0.989
ann_only	0.787	0.907
phonetic_only	0.736	0.882
sn_only	0.713	0.841
housenum_only	0.665	0.800

Each individual blocker only reaches 0.66-0.79 recall alone; the union reaches 0.969 — the four blockers are meaningfully complementary, not redundant.

7. Feature Engineering (src/ber/features.py)

Computed for every (S1, candidate) pair that survives blocking. 41 features feed the model (see `FEATURE_COLUMNS` in the code for the exact current list — two were added and then reverted; Section 8.2 explains why).

7.1 Name features

- RapidFuzz Levenshtein ratio, Jaro-Winkler, `token_set_ratio`, `token_sort_ratio`, `partial_ratio` on `name_core` (and a best-of variant against `name_alt` for dba cases).
- TF-IDF cosine (reused from the blocking re-rank stage), exact-match flags (core and full), first-token match, token Jaccard.
- Legal-form agreement: match / conflict / missing, computed from the `legal_forms` set on each side.
- `is_domain` flag, translit flag (was either side originally non-ASCII), name length ratio.

7.2 Address features

- RapidFuzz ratio and `token_set_ratio` on the full normalized address; token Jaccard and numeric-token Jaccard on the structured pieces.
- House-number, pincode, and state match flags, each coded 1 / 0 / -1 (match / mismatch / missing-so-no-evidence).
- Landmark token overlap (-1 when neither side has one), empty-address flags for each side, address length ratio.

7.3 Other features

- Country exact-match flag (no one-hot encoding of country anywhere — France appears only in test and must be handled by the same generic features as US/India).

- Blocker provenance flags, embedding cosine, ANN rank, TF-IDF re-rank score, source (S2 vs S3).
- The competition features from Section 6.3 (n_competitors, rank_for_cand, gap_to_best_for_cand, rank_for_s1, gap_to_best_for_s1).

7.4 A real performance finding: jaccard/legal-form computation must not build persistent Python object columns

The first implementation built full-length pandas Series of Python frozenset objects (one per jaccard-based feature: name tokens, address tokens, numeric tokens, landmark tokens) and kept several of them alive simultaneously. On the full candidate table this drove system memory from 15.7GB down to ~1GB free (confirmed) — a real near-OOM, not just "slow". Rewritten to compute each row's two sets, use them, and discard them within one tight loop per pair (never retained in a persistent column), chunked and parallelized across processes the same way the RapidFuzz scoring already was. Net effect: bounded, predictable memory regardless of table size, and faster besides (previously unparallelized).

8. The India/US Gap: Diagnosis, Two Attempted Fixes, Both Reverted

This is the most important section for whoever picks this up next. We found a real, well-quantified problem, tried two targeted fixes, and both made the measured score worse on rigorous testing. The diagnosis is solid; the fixes were not. Do not re-attempt either fix exactly as described below without a new idea for why it would behave differently.

8.1 The diagnosis

On the dry-run validation split: **India ceiling F0.5 = 0.9749, final F0.5 = 0.9075 vs US ceiling F0.5 = 0.9953, final F0.5 = 0.9597**. Both the blocking ceiling and the final score trail meaningfully for India, and the ceiling-to-final gap is proportionally worse for India too (India's classifier captures ~93.1% of its own ceiling; US's captures ~96.4%) — so it's not purely a blocking problem.

Pulled 15-20 India false negatives and split them into "never became a candidate" (blocking miss, 47.6% of India's FNs) vs "scored below threshold" (classifier miss, 52.4%). Findings:

- **Blocking misses are dominated by transliteration.** Native-script names are 23.4% of all India S2/S3 records but **63.9%** of blocking-miss false negatives — a 2.7× over-representation. Example: anyascii turns "White Industries" (■■■■■■■■ ■■■■■■■■■■■■) into vhait imdstrij — phonetically unrecognizable, which breaks metaphone-based phonetic blocking outright, and plausibly hurts the embedding blocker too (a multilingual embedding model is trained to align meaning *across* scripts, but we feed it already-mangled ASCII instead of the original script it would understand well).
- **Classifier misses ("scored below threshold") are dominated by missing addresses**, not house-number-format issues: empty/near-empty addresses are 24.7% of these FNs vs a 3.4% baseline rate (7.3× over-representation). Concrete example: a perfect name match (name_token_set = 1.000) with an empty address on one side crashed to calibrated probability 0.0057.
- A smaller structural/ground-truth-noise contributor: digit-level corruption in the raw address itself (house number "11" vs "1" from "E-11/8" vs "E-1/8"), and Telangana/Andhra Pradesh state-code disagreement for the same city (a real historical bifurcation, both technically defensible) — irreducible given the input data.

8.2 Fix attempt 1: an interaction feature for "high name confidence, address absent" — REVERTED

Added either `_addr_missing` (flag) and `name_conf_addr_missing` (= `name_token_set`, gated to nonzero only when either address is missing) so the model could learn to trust the name more when address evidence simply isn't available, instead of treating "missing" the same as "mismatched".

Result on the dry run: F0.5 dropped from 0.9390 to 0.9249, and the model started slightly *underperforming* the simple baseline (-0.0015 vs the previous +0.0126). Ablation testing (retraining on the same cached features with different column subsets) found something non-obvious: dropping *either one* of the two new columns alone was worse (0.9227 / 0.9217) than dropping *both* (0.9358) — a harmful interaction between two highly-correlated columns, not one simply-bad feature. Even "both dropped" (0.9358) didn't fully recover the original 0.9390, likely from the reshuffled hard-negative sampling composition this change caused as a side effect.

Current state: fully reverted, removed from `FEATURE_COLUMNS` and from `compute_features`. The diagnosis (missing address should not be treated as evidence of mismatch) is still believed correct. **Suggested next approach for whoever picks this up:** the code comment in `features.py` points at option (b) from the original plan — fix this via *hard-negative sampling* (ensure training negatives include high-name-similarity/missing-address true-negatives) rather than a new column — untried.

8.3 Fix attempt 2: embed native-script text directly instead of transliterating first — DISABLED (toggleable)

Built an `_EmbedIndex` class (`blocking.py`) that, for any record with genuine non-ASCII content, embeds *both* the transliterated text and a lightly-cleaned native-script version, treating them as independent rows in the same FAISS index/query set. Because the union's per-pair aggregation already takes the **max** similarity across any duplicate blocker hits, this gives "take the max similarity across both representations" automatically, with no special two-vector search path needed.

Gated behind a config flag: `cfg.use_native_embedding` (env var `BER_USE_NATIVE_EMBED`, default **0/False**).

Result: the blocking ceiling barely moved for India (0.9358→0.9365 recall, +0.0007) — essentially flat, not the expected fix — while overall F0.5 also regressed once combined with fix attempt 1 above (0.9249). After reverting fix 1 and isolating fix 2 alone: F0.5 = 0.9362, still short of the 0.9390 bar, and **India's F0.5 stayed at 0.9025 — barely moved from 0.9075**. The embedding fix did not close the gap it was diagnosed to address, and introduced a small residual drag (most likely by reshuffling which candidates rank highest, which changes hard-negative sampling composition even though the direct ceiling effect is small).

Current state: the code is intact and fully toggleable (set `BER_USE_NATIVE_EMBED=1` to re-enable), but disabled by default. The bucket cap and cache-tag-suffixing (`_asciionly` vs `unsuffixed`) both correctly handle the toggle without cache collisions.

8.4 What we did NOT get to try

- Hard-negative sampling redesign to specifically include high-name-similarity/missing-address negatives (the deferred option for fix attempt 1).
- A genuinely separate ablation of the embedding fix's effect on hard-negative composition, independent of any feature change — we ran out of dry-run iteration time before isolating this cleanly.
- Any France-specific validation, since France has no training labels at all — the pipeline treats country as a fully open set (no one-hot encoding anywhere) specifically so it degrades gracefully to unseen countries, but this is untested by construction, only reasoned about.

9. Model, Threshold Tuning, and Output (Sections 4-6 of the original plan)

9.1 Split

Deterministic hash of each S1's entity_id (stable across runs, unlike Python's randomized hash()) assigns every S1 — and every candidate pair for it — to exactly one pool: **val 15%**, **calib 5%**, **train** (remaining 80%, optionally subsampled to a configurable cap). Validation ground truth never touches training, calibration, or IDF fitting.

9.2 Labeling and training

- Positives: ground-truth pairs present in the final candidate set. **Hard negatives**: the highest-ranked (by the competition features) non-matching candidates per S1, capped at 12 — not random negatives.
- XGBoost hist, device="cuda" when available with automatic CPU fallback (same algorithm either way, so results are consistent across machines).
- Isotonic regression calibrates raw XGBoost probabilities, fit on the held-out calib pool.

9.3 Assignment and threshold tuning (src/ber/tune.py)

One-owner assignment happens before thresholding: each candidate goes to whichever S1 scored it highest (ties by embedding cosine, then TF-IDF). Because the ground truth never gives one candidate to two different S1s (Section 3.2), this ordering can only ever remove a false positive from a losing S1, never remove a true positive from the winner — so it's at least as good as thresholding first and resolving collisions after, and it lets the entire threshold sweep run as vectorized array ops.

- **Decision mode A — global threshold**: one probability cutoff, swept over ~190 values, chosen to maximize F0.5. Implemented with np.bincount over a once-factorized array rather than one pandas groupby per threshold — the naive version was, at full scale, *slower than training the model itself* (confirmed and fixed; see Section 10).
- **Decision mode B — per-entity expected-F0.5 maximization**: for each S1, sort its candidates by calibrated probability and pick the prefix length (including zero, i.e. singleton) that maximizes an expected F0.5 treating each candidate's probability as its true-match likelihood. Needs no threshold; adapts per entity.
- Both modes are evaluated on validation and whichever scores higher is used. An **honest 2-fold cross-fit** estimate (tune on one half of validation, score on the other, swap, average) is also reported alongside the in-sample number.

9.4 Output and validation

- matching_results.tsv / candidate_pairs.tsv written per the exact spec: one row per required S1, comma-joined IDs, empty for no match, deduplicated, S2-/S3- IDs only, UTF-8, **"\n"-only line endings** (the official validator splits on "\n" only — pandas' Windows default of "\r\n" would silently corrupt every ID in the last column; this is handled centrally in io.py).
- The organizers' own validate_submission.py is invoked as a subprocess for the **test** split (it hardcodes test_source1/2/3.tsv, so it structurally cannot validate the train split's held-out output). For train, validate_predictions.py (or an inline cross-check via ber.metrics.f05_macro) recomputes the score independently from the written files.

10. Bugs Found and Fixed Along the Way

Listed because several of these are non-obvious and could easily be re-introduced or re-discovered the hard way. All are fixed in the current code.

10.1 HuggingFace tokenizer deadlock (silent hang, zero CPU, zero error)

Loading the CUDA sentence-transformer model, then later calling joblib for parallel CPU work (any backend — both multiprocessing/loky and threading were confirmed to hang) deadlocks indefinitely, with no exception and no CPU usage — indistinguishable from a stalled process without deliberately investigating. Root cause:

HuggingFace's fast (Rust/Rayon) tokenizer keeps its own internal thread pool once used, and explicitly does not tolerate more parallelism starting elsewhere in the same process afterward. **Fix:**

TOKENIZERS_PARALLELISM=false, set in config.py before any model import happens anywhere in the codebase.

10.2 FAISS IVF training was oversized regardless of index size

K-means clustering (not add()) dominates FAISS IVF build time, and its cost is set by the *training sample size*, not the corpus size. The original code trained on a flat 300,000-point sample regardless of nlist — FAISS itself warned this was ~3× more than its own recommended minimum. Measured: 88 seconds to cluster 300k points into 2,828 centroids vs. ~9 seconds at a properly-sized ~50k sample, no measurable quality loss for this task.

Fix: training sample capped near FAISS's own recommended minimum (~40 points/centroid); niter reduced from the default 25 to 12 for the same reason.

10.3 House-number-only blocking was >25x slower than phonetic blocking

A bare house number like "1" or "100" is common enough on its own to create enormous buckets. **Fix:** combined with state in the bucket key, matching how phonetic blocking already combines state with metaphone — brought cost back in line (1.76s vs phonetic's 0.60s on the same sample, from 15.10s beforehand).

10.4 Feature-engineering memory blowup

Covered in Section 7.4 — building multiple persistent pandas Series of Python frozenset objects drove system memory from 15.7GB to ~1GB free. Fixed by never retaining the intermediate sets past one row's computation.

10.5 Unvectorized threshold sweep dominated tuning time

~190 thresholds × a full groupby(...).apply(set) reconstruction each, run **four times over** (the main sweep, both cross-fit folds, and the baseline sweep) — at full scale this became slower than training the XGBoost model itself.

Fix: rewritten with np.bincount over a once-factorized, once-filtered array. Verified numerically identical to the naive version on a synthetic test (max absolute difference: float epsilon) before shipping.

10.6 Official validator subprocess crash on Windows (encoding)

The validator prints an em-dash; a piped (non-console) child process on Windows falls back to the system codepage (cp1252) for stdout unless PYTHONIOENCODING forces otherwise, and our UTF-8 decode then choked on the resulting byte, crashing the subprocess's own output-reader thread before we ever saw a return code (surfacing as TypeError: unsupported operand type(s) for +: 'NoneType' and 'str'). **Fix:** force PYTHONIOENCODING=utf-8 in the child's environment, decode with errors="replace", and guard against either stream being None.

10.7 stage_tune didn't force a stale intermediate cache to refresh after a model retrain

stage_tune called its own scoring helper with force=False hardcoded, regardless of the CLI's own --force flag. Retraining the model with different features and then re-running --stage tune --force silently reproduced the *old*

model's exact numbers, because tuning read the stale scored/assigned cache instead of re-scoring with the freshly retrained model. Confirmed by reproducing identical output before and after retraining. **Fix:** force now passed through to the scoring helper.

10.8 HashingVectorizer OOM at full scale (found during the full run)

HashingVectorizer.transform() allocates one internal buffer sized for its *entire* input batch. The train split's IDF-fitting sample is ~5.2M documents passed in a single unchunked call — at full scale this failed a 4GiB allocation under real memory pressure (numpy._core._exceptions._ArrayMemoryError). The dry run's much smaller sample never reached this scale, so it was invisible until the actual full run. A built-in one-shot retry happened to succeed on a second attempt (large-allocation failure is somewhat transient), but the underlying fragility was real. **Fix:** fit_idf now transforms in bounded chunks (200k documents) and stacks the results — verified to produce numerically identical IDF weights to the unchunked version, so this is a pure robustness fix with zero behavioral change.

10.9 A process-management incident (environment-specific, not a code bug)

A malformed background-launch shell command (an unmatched parenthesis) partially executed and left an un-killed parent process running; a later, correctly-relaunched attempt then raced against it, and both wrote to the same cache/output directories concurrently. Root cause was compounded by two Windows/Git-Bash-specific surprises worth knowing about if anyone else automates long background runs on this same kind of setup: (1) a cross-process kill -0 \$PID-based completion watcher fired a premature "done" marker while the real work was still running — PID tracking across nohup'd bash processes proved unreliable here; (2) what looked like a suspicious "venv python spawning system python" process pair is actually normal — this venv's python.exe is a thin redirector into the real interpreter, so one logical process always shows as a parent+child pair. **Resolution:** killed everything, wiped all cache/output state as untrustworthy, and relaunched with a single self-contained script (no cross-process watching at all) that signals completion with a literal ALL_DONE string appended to its own log at the very end — a plain file-content check, not a process-liveness assumption.

11. Dry-Run Methodology and Final Confirmed Numbers

Before committing to the ~13-15 hour full-scale run, we built make_dryrun_subset.py: samples 5% of S1 entities, keeps every one of their true matches (looked up from ground truth) so blocking recall is meaningful rather than artificially near-zero, plus a proportional 5% random background sample of S2/S3 as distractors. Resulting sizes: 110,341 S1 / 426,932 S2 / 451,697 S3 — about 8% of the full data by record count. This let us iterate in ~25-45 minutes instead of hours per attempt.

11.1 Final confirmed dry-run numbers (reverted/known-good configuration)

Metric	Value
Overall F0.5 (in-sample tuned)	0.9390
Overall F0.5 (2-fold cross-fit, honest)	0.9391
Blocking recall ceiling (pair recall / ceiling F0.5)	0.9689 / 0.9886
Simple weighted baseline F0.5 (0.40 name + 0.35 addr + 0.15 housenum + 0.10 country)	0.9264
GBM improvement over baseline	+0.0126
Chosen decision mode	global_threshold, t ≈ 0.51 (varies slightly run to run)
Threshold sensitivity (t ± 0.05 / ± 0.10)	F0.5 stayed in 0.937-0.939 — not brittle

Metric	Value
Singletons in val split	910 / 16,677 (5.5%) — 843 correctly predicted empty (92.6%)
India final F0.5	0.9075
US final F0.5	0.9597

The near-identical in-sample vs. cross-fit numbers (0.9390 vs 0.9391) indicate the tuning process is not overfitting to the validation split — a meaningfully different number here would have been a red flag.

12. Current Status: Full-Scale Run In Progress

What's running right now, locally, using the confirmed-good reverted configuration
(BER_USE_NATIVE_EMBED=0, the missing-address feature already removed from features.py):

```
cd business_entity_resolution
export BER_DATA_DIR="./student_resource/dataset" BER_CACHE_DIR="cache" \
    BER_OUTPUT_DIR="output" BER_USE_NATIVE_EMBED=0
./venv/Scripts/python.exe -u run_pipeline.py --split train --stage all --allow-long
./venv/Scripts/python.exe -u run_pipeline.py --split test --stage all --allow-long
./venv/Scripts/python.exe -u ../student_resource/utis/validate_submission.py \
    --matching output/matching_results.tsv --candidate output/candidate_pairs.tsv \
    --test-dir ../student_resource/dataset/test --check-ids
```

Launched as a single self-contained detached script (run_full.sh, one automatic retry per split), logging to full_run.log. **Check progress with:** tail full_run.log. **Check completion with:** the literal string ALL_DONE appearing as the last line of that file — do not rely on any separate marker file or process-liveness check (see the incident in Section 10.9).

12.1 Timing

Stage	Local estimate per split
Embedding (GPU)	~2.7-4h (train's actual run included one crash+retry; see 10.8)
TF-IDF blocking (CPU FAISS, no Windows GPU wheel)	~1h
Phonetic + house-number + sorted-neighborhood	~3-5 min
TF-IDF re-rank (11 parallel CPU workers)	~1.5-2h
Featurize (RapidFuzz, 11 workers)	~15-20 min
Train + tune + predict + validate	~15-30 min
Total per split	~6.3h
Train + test combined	~13-15h (started this morning; expect train's validation F0.5 available mid-afternoon, full test predictions by evening)

12.2 Cloud alternatives considered and why we stayed local

Investigated Kaggle, Colab, and AWS as parallel/backup options. Conclusion for our specific workload: **free-tier cloud is not clearly faster, and may be slower.** The pipeline's biggest cost is split between a GPU-bound stage (embedding) and a CPU-parallel stage (TF-IDF re-rank + featurize, both joblib-parallelized). Kaggle/Colab free tier give a much better GPU than our local RTX 2050, but only ~4 vCPUs vs our local 11 — the CPU-bound stage would get *slower*, not faster, likely outweighing the GPU win (estimated ~8-10h/split on Kaggle vs our measured ~6.3h/split locally). AWS (via a real EC2 instance like g5.4xlarge, ~16 vCPU + a fast GPU) would likely be the

fastest option of the three since you can pick both a strong GPU *and* enough cores, estimated ~3-3.2h/split (~\$10-15 in compute) — but requires an actual AWS account with a payment method on file, which the team chose to skip for now given time pressure and payment-setup friction. This reasoning is recorded here so it isn't re-litigated without new information (e.g. if someone gets a no-payment-required GPU credit, or if the CPU-bound stages get further optimized/vectorized, the calculus changes).

13. Open Items — Where to Pick This Up

- **Close the India/US gap** (Section 8) without regressing overall score. Leading unexplored idea: fix the missing-address problem via hard-negative sampling (include high-name-similarity/missing-address true-negatives in training) rather than a new feature column. Also worth isolating the embedding fix's effect on hard-negative composition specifically, independent of any feature change, before writing it off entirely.
- **Adaptive k_final**: every S1 currently gets exactly the same k_final=40 candidates regardless of how many it actually needs (confirmed: 100% of val entities hit the cap post-truncation) — diluting simple/unambiguous entities with near-certain negatives. A per-entity adaptive cutoff (e.g. based on the score gap after the top candidates) is untried.
- **Isotonic calibration quality near the decision threshold**: the dry run's calib pool is small (5% of a 5% subsample). Some flat-spot/steep-compression behavior was observed near the threshold on the dry run; expected to improve with the full run's ~20× larger calib pool, but not yet independently confirmed — worth checking once the full run's numbers are in.
- **France** (test-only, no training labels) is handled only by the general "no country one-hot" design principle, never empirically validated since there's no ground truth to check against. Worth a manual spot-check of France predictions once test output exists (the pipeline already prints a per-country match-rate sanity check for exactly this reason).
- **GPU-accelerated re-rank**: TF-IDF re-rank is the single biggest CPU-bound cost. If pursued on a GPU-rich environment (e.g. AWS), moving this to a GPU-friendly embedding-based re-rank instead of CPU sparse TF-IDF could meaningfully change the stage-cost balance discussed in Section 12.2.
- The methodology document (Documentation_template.md at the challenge root) still needs filling in for the final submission package — this handover document has essentially all the raw material for it.

14. How to Reproduce / Run

14.1 Local setup

```
cd business_entity_resolution
python -m venv .venv
.venv\Scripts\activate
pip install torch==2.7.1 --index-url https://download.pytorch.org/whl/cu128 # or CPU-only build
pip install -r requirements.txt
python -m pytest tests/ -q # should show 7 passed
python run_pipeline.py --split train --stage benchmark # ALWAYS run this before a long stage
python run_pipeline.py --split train --stage all --allow-long
python run_pipeline.py --split test --stage all --allow-long
```

14.2 Fast iteration (dry run)

```
python make_dryrun_subset.py --frac 0.05 --data-dir ../student_resource/dataset --out-dir dryrun_data
BER_DATA_DIR=dryrun_data BER_CACHE_DIR=cache_dryrun BER_OUTPUT_DIR=output_dryrun \
```

```
python run_pipeline.py --split train --stage all --allow-long
```

14.3 Kaggle / Colab

See `notebook_cells.py` (paste each `# %% [n]` block into its own cell) or run `make_notebook.py` to generate `pipeline.ipynb` directly. `README.md` has the exact environment-variable settings for each platform.

14.4 Re-enabling a reverted experiment

- Native-script embedding: set `BER_USE_NATIVE_EMBED=1` (or `cfg.use_native_embedding=True`).
- Missing-address feature: the code was fully removed, not just disabled — see the git history / this document's Section 8.2 for the exact feature definitions to reintroduce if trying a different variant.

Appendix: Key Files Quick-Reference

File	What it's for
src/ber/config.py	All tunable knobs, env-var overridable, GPU/CPU auto-detect
src/ber/lexicons.py	Every hand-authored normalization dictionary
src/ber/normalize.py	Section 5
src/ber/blocking.py	Section 6, incl. the _EmbedIndex native-embedding toggle
src/ber/features.py	Section 7, incl. FEATURE_COLUMNS (the current, post-revert list)
src/ber/model.py	Labeling, hard negatives, XGBoost, calibration
src/ber/tune.py	Assignment + both decision modes + the vectorized sweep
src/ber/metrics.py	The exact F0.5 formula, single source of truth
src/ber/pipeline.py	Stage orchestration and all on-disk caching
run_pipeline.py	CLI entry point
validate_predictions.py	Standalone scorer against a held-out split
make_dryrun_subset.py	Builds the 5% fast-iteration subsample
run_full.sh	The exact script currently running the full train+test job
tests/test_smoke.py	Normalization + metric formula unit tests